



Fundamentos de Sistemas Embebidos

Facultad de Ingeniería

UNAM

Laboratorio Salas A y B

Profesor(a): Eduardo Fragoso Navarro

Asignatura: Fundamentos de Sistemas Embebidos

Grupo: 3

No de Práctica(s): 3

Integrante(s): Tepal Briseño Hansel Yael - 320078282

Ugartechea González Luis Antonio - 320223822

No. de brigada: 12

Semestre: 9°

Fecha de entrega: 17 de septiembre de 2026

Observaciones: _____

CALIFICACIÓN: _____

Núcleos y Prioridades

Objetivo

Comprender y aplicar los conceptos de multitarea en sistemas embebidos utilizando el sistema operativo en tiempo real FreeRTOS sobre la tarjeta de desarrollo ESP32-S3. Se busca implementar y analizar el comportamiento de múltiples tareas simultáneas asignando distintos niveles de prioridad y afinidad de núcleos, visualizando el impacto en el rendimiento al procesar animaciones gráficas en una pantalla OLED. Asimismo, se emplean entradas analógicas para controlar dichas tareas dinámicamente y determinar el foco de atención del sistema operativo.

Resultados y Análisis

Ejercicio 1 y 2: Circuito y Programa Base

Se implementó físicamente el circuito en base al esquema de la figura 1 conectando los dos potenciómetros a los pines de entrada analógica de la tarjeta (ADC1) y la pantalla OLED SSD1306 a los pines designados para el bus I2C.

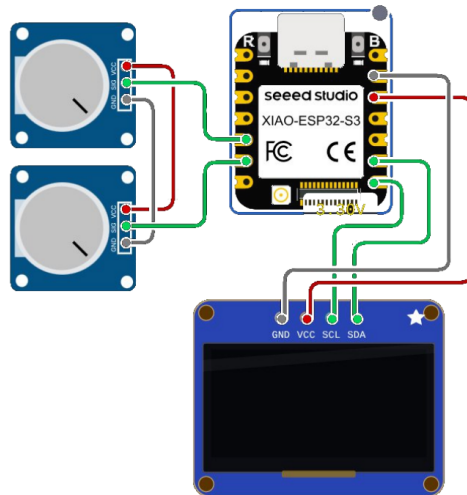


Figure 1: Esquema de conexión del circuito con dos potenciómetros y pantalla OLED.

En el código base del programa principal, la carga de procesamiento intensivo se encuentra dentro del módulo gráfico, cuyo costo computacional aumenta drásticamente en función del número de esferas (N_ESFERAS) y la cantidad de rayos por píxel para el renderizado (SUPER). Dado que el microcontrolador no posee la capacidad de procesar ambas ventanas a máxima fluidez de forma paralela si operan en el mismo núcleo físico, el programa se divide en tres tareas para gestionar

los recursos: vista1 (dibuja la mitad izquierda), vista2 (dibuja la mitad derecha) y control (monitorea los potenciómetros y distribuye prioridades).

Lo más pesado se concentra en la función `grafico_calcular`, que cada tarea de vista ejecuta continuamente para generar su mapa de píxeles. En cambio, la tarea `control` se encarga de leer los potenciómetros y actualizar las prioridades, por lo que se ejecuta en un núcleo independiente. El siguiente fragmento resume la configuración principal del programa:

```
1 #define CORE_VISTA1 0
2 #define CORE_VISTA2 0
3 #define CORE_CONTROL 1
4
5 static void vista1(void *pv)
6 {
7     while (1) {
8         grafico_calcular(mapa1[1 - listo1], valor1);
9         listo1 = 1 - listo1;
10    }
11 }
12
13 xTaskCreatePinnedToCore(vista1, "vista1", STACK, NULL, 1,
14                        &h_vista1, CORE_VISTA1);
15 xTaskCreatePinnedToCore(control, "control", STACK, NULL, 5,
16                        &h_control, CORE_CONTROL);
```

Listing 1: Distribución de las tareas y carga de procesamiento

Como se observa en el fragmento de código 1, las dos tareas de renderizado comparten el Núcleo 0 y realizan una operación costosa dentro de un ciclo infinito. La tarea de control se fija al Núcleo 1 y comienza con una prioridad mayor, lo que le permite seguir leyendo las entradas y reorganizando las prioridades aunque el renderizado esté utilizando intensivamente el Núcleo 0.

Ejercicio 3: Parámetros, Tweaks y Observación del RTOS

La fase analítica de la práctica recae en el comportamiento del planificador al modificar la matriz de prioridades `PRIO[N_SIT]` y las asignaciones de núcleos. Según el código analizado, las tareas gráficas (`vista1` y `vista2`) fueron forzadas a compartir el Núcleo 0, mientras que la tarea de control se aisló en el Núcleo 1.

Dependiendo del movimiento de las perillas, se observaron las siguientes situaciones:

- **Estado NADIE:** Ningún potenciómetro se mueve. Ambas tareas gráficas adquieren prioridad 1.

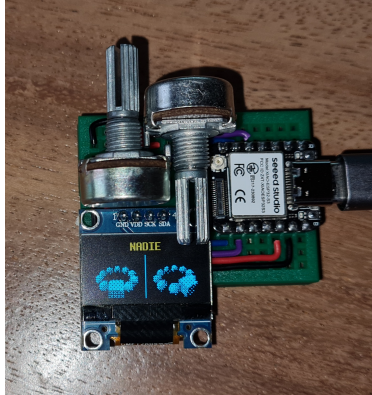


Figure 2: Estado NADIE (Animaciones en reposo).

- **Estado AMBAS:** Al accionar ambas perillas, vista1 y vista2 retienen la prioridad 1. Esto provoca que compitan por el tiempo de CPU del Núcleo 0 dividiendo los recursos por igual, resultando en una pérdida de fotogramas (stuttering) en las dos animaciones simultáneamente.

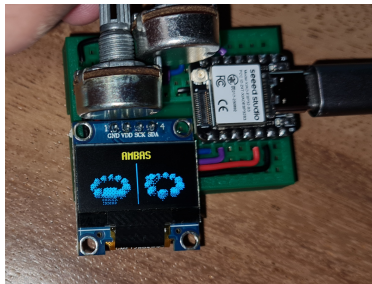


Figure 3: Estado AMBAS (Competición de CPU).

- **Estado SOLO1:** Al mover exclusivamente el primer potenciómetro, el programa altera la tabla elevando la prioridad de vista1 a 3 y degradando vista2 a 1. Al tener mayor prioridad, FreeRTOS otorga el uso exclusivo del núcleo a la ventana izquierda, la cual responde con total fluidez, mientras la derecha se congela o retrasa severamente.

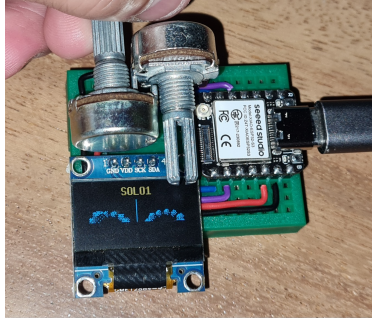


Figure 4: Estado SOLO1 (Foco en renderizado izquierdo).

- **Estado SOLO2:** De forma inversa, si se acciona el segundo potenciómetro, vista2 adquiere prioridad 3 y vista1 baja a 1, concentrando el poder de cómputo en la ventana derecha para hacerla inmediata y fluida.



Figure 5: Estado SOLO2 (Foco en renderizado derecho).

Análisis de Resultados y Experimentos

Prioridades

La siguiente tabla muestra una variedad de situaciones en las que se hicieron modificaciones unicamente a las prioridades de las tareas, manteniendo la configuración de núcleos original.

Tarea	Prioridad vista1	Prioridad vista2	Prioridad control	Core vista 1	Core vista 2	Core control
Situación 1						
SOLO1	1	1	5			
SOLO2	1	1	5			
AMBAS	1	1	5	0	0	0
NADIE	1	1	5			
Esta es la situación base, en la que se puede ver que las prioridades están completamente distribuidas hacia la tarea de control, al igual que la distribución de los núcleos. Esto provoca que las tareas de renderizado compitan por el tiempo de CPU y se genere “stuttering” en ambas animaciones.						
Situación 2						
SOLO1	3	1	5			
SOLO2	1	3	5			
AMBAS	1	1	5	0	0	0
NADIE	1	1	5			
En esta situación, se observa que al mover un potenciómetro, la tarea correspondiente a este potenciómetro adquiere una mayor prioridad. Esto provoca que el planificador de FreeRTOS le otorgue un uso más enfocado del núcleo, mientras que la otra tarea de renderizado se queda con prioridad baja y no puede competir por el tiempo de CPU. Como resultado, la animación asociada al potenciómetro en movimiento es fluida, mientras que la otra se congela o se retrasa. Este comportamiento aplica para cualquier prioridad mayor que 1, con excepción de la prioridad 5, en cuyo caso se produciría una competencia entre la tarea de control y la tarea de renderizado que podría afectar la actualización de la pantalla.						
Situación 3						
SOLO1	5	1	3			
SOLO2	1	5	3			
AMBAS	1	1	3	0	0	0
NADIE	1	1	3			
En este caso, al mover un potenciómetro, la tarea correspondiente adquiere una prioridad mayor que la de control. El planificador de FreeRTOS le otorga entonces un uso más enfocado del núcleo, mientras que la otra tarea de renderizado conserva una prioridad baja y no puede competir por el tiempo de CPU. La animación asociada al potenciómetro en movimiento se mantiene fluida, pero la tarea de control puede ejecutar menos lecturas, lo que ocasionaría un retraso en la respuesta ante cambios de posición. Por lo tanto, aunque aumentar la prioridad de renderizado mejora la fluidez, reducir demasiado la prioridad de control puede perjudicar la interacción del sistema con el usuario.						

Table 1: Comparación de situaciones con distintas prioridades

Conclusión: Con respecto a las situaciones planteadas en la tabla 1, Se puede concluir que la prioridad de la tarea de control debe mantenerse alta para garantizar una respuesta rápida a las entradas del usuario. Por otro lado, aumentar la prioridad de las tareas de renderizado puede mejorar la fluidez de las animaciones, pero si se aumenta demasiado puede generar retrasos en la lectura de los potenciómetros debido a la pelea por el tiempo de CPU. Por lo tanto, es importante encontrar un equilibrio adecuado entre la prioridad de las tareas de renderizado y la

tarea de control para lograr un rendimiento óptimo del sistema. De igual forma la optimización de prioridades de acuerdo a la situación activa permite que el sistema se enfoque en la tarea que requiere más atención.

Es por esto que decidimos que la mejor situación para distribuir bien las prioridades es la situación 2, en la que la tarea de control mantiene una prioridad alta y las tareas de renderizado pueden aumentar su prioridad solo cuando se mueven los potenciómetros, lo que permite un enfoque dinámico en la tarea que requiere más atención.

```
1 static const prioridades_t PRIO[N_SIT] = {  
2     /* SOLO1 */ {3, 1, 5},  
3     /* SOLO2 */ {1, 3, 5},  
4     /* AMBAS */ {1, 1, 5},  
5     /* NADIE */ {1, 1, 5},  
6 };
```

Listing 2: Distribución de prioridades en la situación 2

Afinidad de Núcleos

La siguiente tabla muestra una variedad de situaciones en las que se hicieron modificaciones unicamente a la afinidad de núcleos de las tareas, manteniendo la configuración de prioridades que nosotros establecimos como óptimas.

Tarea	Prioridad vista1	Prioridad vista2	Prioridad control	Core vista 1	Core vista 2	Core control
Situación 1						
SOLO1	3	1	5			
SOLO2	1	3	5	0	0	1
AMBAS	1	1	5			
NADIE	1	1	5			
En esta situación, las tareas de renderizado comparten el Núcleo 0, mientras que la tarea de control se ejecuta en el Núcleo 1. Esto permite que la tarea de control tenga un tiempo de CPU dedicado y pueda responder rápidamente a las entradas del usuario, mientras que las tareas de renderizado compiten por el tiempo de CPU del Núcleo 0. Como resultado, la animación se nota más responsiva y fluida cuando solo se mueve un potenciómetro, mientras que si se mueven las dos sigue siendo responsiva, pero con un ligero retraso en la actualización de la pantalla.						
Situación 2						
SOLO1	3	1	5			
SOLO2	1	3	5	0	1	1
AMBAS	1	1	5			
NADIE	1	1	5			
En esta configuración, cada tarea de renderizado se ejecuta en un núcleo diferente y la tarea de control comparte el Núcleo 1 con vista2. Esto permite que ambas animaciones se calculen en paralelo, aunque la ventana derecha puede competir parcialmente con la lectura de los potenciómetros. El resultado esperado es una mejora importante cuando ambas perillas se mueven, pero con una posible reducción en la capacidad de respuesta de la tarea de control.						
Situación 3						
SOLO1	3	1	5			
SOLO2	1	3	5	1	0	1
AMBAS	1	1	5			
NADIE	1	1	5			
Esta situación es el caso inverso de la anterior: vista1 comparte el Núcleo 1 con control, mientras que vista2 utiliza el Núcleo 0 de forma independiente. La animación derecha puede mantenerse estable, pero la respuesta de la ventana izquierda y la lectura de los potenciómetros pueden verse afectadas cuando aumenta la carga de renderizado en el Núcleo 1.						
Situación 4						
SOLO1	3	1	5			
SOLO2	1	3	5	0	0	0
AMBAS	1	1	5			
NADIE	1	1	5			
Finalmente, en esta configuración las tres tareas comparten el Núcleo 0. No existe paralelismo entre las tareas de renderizado y, además, control debe competir directamente por el mismo tiempo de CPU. Por ello, esta sería la alternativa menos conveniente: puede aumentar el retraso de las animaciones y provocar que la lectura de los potenciómetros y la actualización de la pantalla sean menos constantes.						

Table 2: Comparación de situaciones con distintas afinidades de núcleo

Conclusión: Con respecto a las situaciones planteadas en la tabla 2, se puede concluir que la

afinidad de núcleos es un factor crítico para el rendimiento del sistema. La situación 1, donde las tareas de renderizado comparten un núcleo y la tarea de control se ejecuta en un núcleo separado, permite una respuesta rápida a las entradas del usuario y una animación fluida cuando solo se mueve un potenciómetro. La situación 2 y 3, donde cada tarea de renderizado se ejecuta en un núcleo diferente y la tarea de control comparte un núcleo con una de ellas, permite que ambas animaciones se calculen en paralelo, aunque puede afectar la capacidad de respuesta de la tarea de control.

Esto nos dió a entender que la mejor situación para distribuir bien las afinidades de núcleos es la situación 1, en la que la tarea de control se ejecuta en un núcleo separado y las tareas de renderizado comparten un núcleo. Esto permite una respuesta rápida a las entradas del usuario y una animación fluida cuando solo se mueve un potenciómetro, mientras que si se mueven ambos potenciómetros, la animación puede experimentar un ligero retraso en la actualización de la pantalla.

```
1 #define SIN_AFINIDAD tskNO_AFFINITY
2 #define CORE_VISTA1 0
3 #define CORE_VISTA2 0
4 #define CORE_CONTROL 1
```

Listing 3: Distribución de afinidades de núcleos en la situación 1

Alternativas de Optimización

El rendimiento también podría mejorarse reduciendo el costo del módulo matemático `grafico.c`; por ejemplo, disminuyendo la cantidad de rayos por píxel mediante `SUPER` o reduciendo el número de esferas definido por `N_ESFERAS`. Estas modificaciones disminuirían el trabajo realizado por cada tarea de renderizado y, por lo tanto, permitirían obtener una mayor frecuencia de actualización.

En la API de FreeRTOS utilizada por la plataforma, la afinidad se establece al crear la tarea mediante `xTaskCreatePinnedToCore`, por lo que el núcleo de ejecución no puede migrarse de forma trivial durante el funcionamiento del programa. La prioridad, en cambio, sí puede modificarse dinámicamente con `vTaskPrioritySet`. Por esta razón, la tarea `control` debe conservar una prioridad alta, en este caso 5, pues es responsable de leer los potenciómetros, identificar la situación activa y redistribuir los recursos. Si tuviera una prioridad baja y compartiera núcleo con las tareas gráficas, estas podrían consumir la mayor parte del tiempo de procesamiento y provocar inanición (*starvation*), dejando al sistema sin capacidad de responder oportunamente a las entradas del usuario.

Conclusiones

El desarrollo de la práctica sirvió como un simulador a microescala del núcleo de un sistema operativo de escritorio convencional. De manera análoga a una interfaz gráfica de computadora donde el sistema asigna la mayor cantidad de ciclos de reloj y prioridad a la ventana que tiene

el foco para garantizar una experiencia de usuario rápida y fluida, en este sistema embebido FreeRTOS concentró los recursos limitados en la tarea gráfica que estaba operándose activamente.

El experimento también demostró el impacto directo que tiene el anclaje físico al procesador. El haber designado desde el inicio que las dos tareas de vista compartieran el mismo núcleo generó un cuello de botella mediante paralelismo real. La manipulación de prioridades solucionó los retrasos para el flujo de interacción focalizada, pero dejó en evidencia la penalización de rendimiento cuando ambas exigen procesamiento conjunto en el mismo hilo.

El comportamiento aislado y privilegiado de la tarea de lectura ilustra un concepto muy vital en la designación de la criticidad del sistema. Así como `control` es invulnerable a la suspensión de CPU generada por las vistas, en el campo industrial existen procesos innegociables (sistemas de frenos ABS, dosificadores médicos, controladores de vuelo o reinicios por watchdog timer) que jamás deben situarse en posiciones de baja prioridad, previniendo riesgos críticos en caso de saturación del microcontrolador.

NOTA: Si se desea consultar el código fuente completo del proyecto, se puede acceder al repositorio de GitHub en la siguiente dirección: https://github.com/lethalSopape/r/laboratorio_sistemas_embebidos